

Lesson 11

by Lorraine Gaudio

Lesson generated on November 01, 2025



BOISE STATE UNIVERSITY

Contents

1	📄 Welcome Back to R!	2
2	📄 Packages	3
3	📄 Building a Pipeline	4
3.1	Start with the data	4
3.2	Create a new variable with <code>mutate()</code>	4
3.3	Keep only cars using greater than 8 L/100 km	5
3.4	Select relevant columns	5
3.5	Group and summarize	5
3.6	Recode transmission for readability	6
3.7	Readable Result	7
4	📄 Assignment	8
4.1	Task 0	8
4.2	Task 1	8
4.3	Task 2	9
4.4	Task 3	12
5	Save and Upload	13
6	Today you practiced:	14

1. 📌 Welcome Back to R!

In lesson 8, 9, and 10, we explored several functions from dplyr (`select()`, `filter()`, `mutate()`, `case_when()`, `group_by` and `summarize()`). You now know the key dplyr verbs one at a time. This week, we'll be putting dplyr verbs together in more complex pipelines. The real power comes from *chaining* them so data flows logically from raw to insight in a single readable statement.

To begin Lesson 11, follow these steps:

1. Open your course project for RStudio
2. Create a new file. From the file types we have used so far, pick which file type you want to use. (File > New File > ???).
3. Type in the code provided in this document as you follow along with the video. Pause the video at anytime to answer assignment questions, dig deeper or add memo notes.

Lesson Overview

By the end of Lesson 11 you will be able to:

1. ☐ Remember – List the actions of main dplyr verbs: `select()`, `filter()`, `mutate()`, `group_by()`, `summarize()`, `arrange()`.
2. ☐ Understand – Describe how the pipe `%>%` passes results step by step.
3. ☐ Apply – Build a multi-verb pipeline to clean and summarize data.
4. ☐ Analyze – Inspect intermediate output to verify each step.
5. ☐ Evaluate – Decide when a single pipeline is clearer than separate objects

Keep these goals in mind as you move through each section.

2. ☒ Packages

Install once (if needed): `install.packages("dplyr"); install.packages("dslabs")`

Load the packages at the start of every session:

```
library(dslabs) # Data science labs package  
library(dplyr)  # Data manipulation package
```

3. ⚡ Building a Pipeline

□ The GOAL: Find the average fuel consumption (liters/100 km) for cars in mtcars that use greater than 8 L/100 km, broken down by transmission and cylinders.

3.1 Start with the data

We'll use the built-in mtcars dataset.

□ Goal. Load mtcars and confirm shape.

```
data("mtcars")  
# □ Test:  
mtcars %>%           # rows = 32 cars, many columns  
  head(2)            # preview first two rows  
  
head(mtcars, 2) # base R version, same output
```

□ Data loaded correctly

3.2 Create a new variable with **mutate()**

We'll calculate fuel consumption in liters per 100 kilometers (L/100 km). The formula is:

$$\frac{\text{L}}{100 \text{ km}} = \frac{235.215}{\text{MPG}}$$

□ Goal. Add l_100km using the equation $235.215/\text{mpg}$ and round the value to one digit.

□ Prediction. Higher mpg → lower l_100km (inverse).

```
# □ Test:  
mtcars %>%  
  mutate(l_100km = round(235.215 / mpg, 1)) %>%  
  head(2)
```

```
# □ Verify:  
cor(mtcars$mpg, 235.215/mtcars$mpg) < 0
```

```
## [1] TRUE
```

□ The inverse units are verified.

3.3 Keep only cars using greater than 8 L/100 km

We'll only keep cars that use *at least* 8 L/100 km. Use `filter()` to keep those rows.

□ Goal. Keep `l_100km >= 8`.

```
# □ Test:
mtcars %>%
  mutate(l_100km = round(235.215 / mpg, 1)) %>%
  filter(l_100km >= 8) %>%
  head()
```

3.4 Select relevant columns

□ Goal. Keep `cyl`, `am`, and `l_100km` columns.

```
# □ Test:
mtcars2 <- mtcars %>%
  mutate(l_100km = round(235.215 / mpg, 1)) %>%
  filter(l_100km >= 8) %>%
  select(cyl, am, l_100km) %>%
  head()
```

```
# □ Verify:
setequal(names(mtcars2 %>% select(cyl, am, l_100km)), c("cyl", "am", "l_100km"))
```

□ Columns trimmed.

3.5 Group and summarize

Now we can group by `am` and `cyl`, then summarize to get the mean fuel consumption and count of cars in each group.

□ Goal. Mean `l_100km` and `n` by `am`, `cyl`.

```
# □ Test:
mtcars_summary <- mtcars %>%
  mutate(l_100km = round(235.215 / mpg, 1)) %>%
  filter(l_100km >= 8) %>%
  select(cyl, am, l_100km) %>%
  group_by(am, cyl) %>%
  summarize(mean_l_100km = round(mean(l_100km), 2),
            n = n(), .groups = "drop")
mtcars_summary
```

□ Explan:

Order Matters in dplyr Pipelines. You can't summarize groups before you define them with `group_by()`.

- `group_by(am, cyl)` tells dplyr to treat each unique combination of `am` and `cyl` separately. `group_by()` divides your data into separate groups for analysis. It doesn't change how your data looks. It adds metadata that tells **subsequent functions** to operate on each group separately. It's like telling R: "For each combination of these variables, do the following..."
- `summarize()` then runs `mean(l_100km)` *per group* and counts rows with `n()`.
- `.groups = "drop"` prevents grouping in the final output.

3.6 Recode transmission for readability

□ Question: How might you check which combination of `am` + `cyl` is least fuel-efficient?

□ Goal. Label `am` as "Automatic/Manual" in a column "Transmission" and make it a factor with level order.

```
# □ Test: Recode transmission for readability Method 1
mtcars_summary <- mtcars_summary %>%
  mutate(Transmission = case_when(am == 0 ~ "Automatic", TRUE ~ "Manual"),
         Transmission = factor(Transmission, levels = c("Automatic", "Manual"))
  )
```

```
# □ Verify:
is.factor(mtcars_summary$Transmission)
identical(levels(mtcars_summary$Transmission), c("Automatic", "Manual"))
```

□ Labels set; order fixed.

☐ Test: ☐ Pipeline complete! One readable flow from raw data to insight.

```
mtcars_summary <- mtcars %>%  
  mutate(l_100km = round(235.215 / mpg, 1)) %>%  
  filter(l_100km >= 8) %>%  
  select(cyl, am, l_100km) %>%  
  group_by(am, cyl) %>%  
  summarize(mean_l_100km = round(mean(l_100km), 2),  
            n = n(), .groups = "drop") %>%  
  mutate(am = case_when(am == 0 ~ "Automatic", TRUE ~ "Manual"),  
         am = factor(am, levels = c("Automatic", "Manual"))  
  )  
mtcars_summary
```

☐ Reflect: Explain why the highest mean_l_100km is the least fuel-efficient combination. Create a memo to earn learning skill points.

3.7 Readable Result

☐ Goal. Build one readable pipeline, then sort by the highest Mean l/100km (higher = less efficient) and keep the top row.

```
mtcars_summary <- mtcars %>%  
  mutate(l_100km = round(235.215 / mpg, 1)) %>%  
  filter(l_100km >= 8) %>%  
  select(cyl, am, l_100km) %>%  
  group_by(am, cyl) %>%  
  summarize(mean_l_100km = round(mean(l_100km), 2),  
            n = n(), .groups = "drop") %>%  
  mutate(am = case_when(am == 0 ~ "Automatic", TRUE ~ "Manual"),  
         am = factor(am, levels = c("Automatic", "Manual"))  
  ) %>%  
  arrange(desc(mean_l_100km)) %>%  
  slice(1) %>%  
  rename(`Mean l/100km` = mean_l_100km,  
         Count = n,  
         Cylinder = cyl,  
         Transmission = am)  
mtcars_summary
```

☐ Explain: arrange() sorts the data frame by mean_l_100km (aka Mean l/100km) in descending order, so the first row is the least fuel-efficient combination of am and cyl.

Answer the ☐ Question: Which combination of am + cyl is least fuel-efficient?

4. ☒ Assignment

Replace each _____ placeholder (and any TODO comments) with working code or a short written answer. Run each section; be sure the requested objects appear in the Environment. When finished, save **BOTH** this script and your .RData workspace and upload to Canvas Assignment 11. Create detailed memos for yourself as you work through each task.

Dataset: dslabs::movielens

There are three tasks. **Task 2** is the main coding task. Drafting steps are *optional* but recommended to help you build the final pipeline and earn **learning skill** points on your process (☐ /☐ Goal → Prediction → ☐ Code → ☐ Checks → ☐ /☐ Reflection). In Task 2, only Movie_Facts is graded for **technical points**. Create a memo about any ☐ Resources you used. ☐ Format your document with section headings, in line text font, and add a date. If you run into issues, ☐ Keep a record of what you tried and portray a ☐ Growth mindset through tenacity and curiosity. Make a ☐ Prediction on what you think the results will be. All these activities demonstrate learning skills.

When you're done, your workspace should contain Movie_Facts that is created by one end-to-end pipeline.

4.1 Task 0

☐ Library it up!

Make sure there is script in your document to load dplyr and dslabs packages so their functions / datasets load.

4.2 Task 1

☐ Quick Recall

Explain how *three* dplyr verbs from Lessons 7-11 work.

☐ EXPLANATION: “_____”

☐ EXPLANATION: “_____”

☐ EXPLANATION: “_____”

4.3 Task 2

□ Pipeline

Use a single pipeline.

Option to draft it in steps outlined below. Only final pipeline is evaluated for **technical skills**.

Drafting steps are for your benefit and give you opportunity to check intermediate results, memo notes, and reflect to earn **learning skill** points. You choose if and when you record □, □, □ / □. *You could pipe head() in for checks instead of having a separate code chunk.*

What your final table must contain

A. Filtering (2.1)

1. Restrict to movies with non-missing rating and year ≥ 1990 .
2. Retain only rating and year columns where the row values are non-missing.

B. The Columns:

1. decade (2.2)

- (a) Calculate the year into decades with decimals (e.g., $1994 \rightarrow 199.4$). Hint: use the equation $\text{year}/10$.
- (b) Round down to nearest 10 (e.g., $199.4 \rightarrow 199$). Hint: use `floor()`. Pass (a)'s equation through `floor()`.
- (c) Convert the count back to decade's starting year (e.g., $(199 \rightarrow 1990)$). Hint: Multiply results by 10.

2. rating_label (2.3)

- (a) Use `case_when()` to create a new column `rating_label` that labels ratings less than 3 as "Bad" and ratings 3 or greater as "Good".
- (b) Make `rating_label` a factor with levels ordered so "Bad" comes before "Good".

3. mean_rating (rounded to 2), (2.4)

- (a) Group by decade and `rating_label`
- (b) Summarize to get **mean** of rating rounded to 2 decimal places.

4. n (row count) (2.5)

- (a) Count of movies in each group (decade and `rating_label`)
- (b) then ungrouped in the result.

C. Ordering:

1. sort by `mean_rating` (descending) or by `n` (you choose; state which you picked in your memo).
 - Store as `Movie_Facts`.

□ Explore and Play: add style when you rename the columns in the final output to be more readable (e.g., Mean Rating, Count). Or add any other enhancements like more levels to rating_label such as “Excellent”. Be sure to point it out in a memo.

4.3.1 Task 2.0

Load packages and confirm movielens has rating and year.

```
# □ Code
```

```
# □ Verify:
```

4.3.2 Task 2.1

Keep only rows rating and year columns when both are non-missing (Hint: use `!is.na()` and `filter()`). Restrict to movies with non-missing rating and year ≥ 1990 .

□

```
# □ Code.
```

```
# □ Verify: years are 1990 or greater
```

□ / □

4.3.3 Task 2.2

Add to the pipeline from Task 2.1, using `mutate`, use `create` the column decade

- Calculate the year into decades with decimals (e.g., $1994 \rightarrow 199.4$). Hint: use the equation $\text{year}/10$.
- Round down to nearest 10 (e.g., $199.4 \rightarrow 199$). Hint: use `floor()`. Pass (a)’s equation through `floor()`.
- Convert the count back to decade’s starting year (e.g., $(199 \rightarrow 1990)$. Hint: Multiply results by 10.

□

```
# □ Code.
```

```
# □ Verify:
```

□ / □

4.3.4 Task 2.3

Continue the same pipeline from Task 2.2, but now add to mutate()

- (a) Use case_when() to create a new column rating_label that labels ratings less than 3 as “Bad” and ratings 3 or greater as “Good”.
- (b) Make rating_label a factor with levels ordered so “Bad” comes before “Good”.

☐

☐ Code.

☐ Verify:

☐ / ☐

4.3.5 Task 2.4

Continue the same pipeline to createmean_rating (rounded to 2),

- (a) Group by decade and rating_label
- (b) Summarize to get **mean** of rating rounded to 2 decimal places. Because NAs are filtered out, na.rm=TRUE isn't needed.

☐

☐ Code.

☐ Verify:

☐ / ☐

4.3.6 Task 2.5

Continue the same pipeline to create n (row count) (2.5)

- (a) Count of movies in each group (decade and rating_label)
- (b) then ungroup the result. (groups = "drop")

☐

☐ Code.

☐ Verify:

☐ / ☐

4.3.7 Task 2.6

Finish the same pipeline with an `arrange()` step (choose to sort by `mean_rating` or by `n`). End by storing the result as `Movie_Facts`.

C. Ordering:

1. sort by `mean_rating` (descending) or by `n` (you choose; state which you picked in your memo).

☐

☐ *Code.*

☐ *Verify:*

☐ / ☐

4.4 Task 3

☐ Write a short paragraph reflecting on when might **separate objects** be clearer than a single long pipeline?

☐ EXPLANATION: “___”

OR

☐ Write a short paragraph reflecting on what `n` is counting in `Movie_Facts`. Is it individual movies or individual ratings? Explain.

☐ EXPLANATION: “___”

5. Save and Upload

1. You will be submitting **both** the “your choice” Document and the workspace file. The workspace file saves all the objects in your environment that you created in this lesson. You can save the workspace by running the following command in a code chunk of the script document document:

```
save.image("Assignment11_Workspace.RData")
```

Or you can click the “Save Workspace” button in the Environment pane.

□ **Always save the R documents before closing.**

2. Find the assignment in this week’s module in Canvas and upload **both** the file with your script and the workspace file.

6. Today you practiced:

- Connected dplyr verbs with the pipe to form readable workflows.
- Built a pipeline that mutated, filtered, selected, grouped, summarized, and recoded in one statement.
- Reflected on the complexity of piping many steps